DMX-Router 4 → 16

Ein konfigurierbarer **4-auf-16 DMX/RDM-Matrix-Splitter** auf Basis eines Raspberry Pi Pico (RP2040).

Die Firmware bietet:

- 4 DMX/RDM-Eingänge (250 kbaud, 8N2) über PIO-UART-Empfänger
- 16 DMX-Ausgänge, parallel über zwei PIO-Bitplane-Sender + DMA
- HTP-Merge (Highest Takes Precedence) pro Ausgang
- best-effort RDM-Store-and-Forward-Relay
- optionale lokale Bedienung über SSD1306-OLED + PCF8574-Tasten
- Konfiguration über USB-Serial-Konsole und LittleFS (`/router.cfg`)
- Konfiguration übersteht Reboots; `defaults` stellt den 4-fach-Splitter wieder her

> **Status:** Der Stand kompiliert vollständig (Standalone- und Root-Env). Ein Test auf realer Hardware (DMX-Pegel, RDM-Timing, Multiplexer) steht noch aus.

---

1. Pin-Belegung

Der RP2040 stellt **30 GPIOs** (GPIO 0..29) bereit. Die Firmware nutzt alle 30 und kommt ohne Portexpander aus:

- Die 16 Ausgänge liegen als zwei zusammenhängende 8-Pin-Gruppen, weil eine PIO-State-Machine mit `out pins, 8` nur einen zusammenhängenden Pin-Bereich treiben kann.
- Die vier Eingangs-Transceiver teilen sich `TXD` und `DIR`; die 16 Ausgangs-Transceiver teilen sich `DIR_OUT`.
- Die vier Tasten liegen am PCF8574 auf dem I2C-Bus und belegen keine eigenen GPIOs.

1.1 Belegung (alle 30 GPIO genutzt)

GPIO	Funktion	Richtung / Hinweis
0..7	**OUT 1..8** (`TX_OUT`)	Ausgang, PIO-Bitplane-Gruppe 0
8..15	**OUT 9..16** (`TX_OUT`)	Ausgang, PIO-Bitplane-Gruppe 1
16..19	**RX_IN 1..4**	Eingang, je ein PIO-UART-Empfänger
20	**TX_IN gemeinsam**	UART1-TX, 250 kbaud 8N2, RDM-Antworten
21	**DIR_IN gemeinsam**	low = Empfang vom Pult, high = RDM-Antwort senden
22..24	**MUX_SEL A/B/C**	Ausgang, gemeinsam für beide 74HC151
25	**DIR_OUT gemeinsam**	high = DMX senden, low = RDM empfangen
26	**MUX_Y1**	Eingang, RDM-Rückkanal OUT 1..8
27	**MUX_Y2**	Eingang, RDM-Rückkanal OUT 9..16
28	**I2C SDA**	OLED + PCF8574
29	**I2C SCL**	OLED + PCF8574

Hinweise:

- **GP25** ist bei einem normalen Pico mit der Onboard-LED verbunden. Die LED hängt hier am gemeinsamen `DIR_OUT` und leuchtet daher im DMX-Sendebetrieb (bei RDM-Empfang kurz aus). Das ist elektrisch unkritisch; wer die LED nicht möchte, kann sie auf der Platine entfernen.
- Die vier Eingangs-`TXD`-Leitungen werden **zusammengefasst** auf GP20 geführt. Eine RDM-Antwort geht damit an alle Pulte; fremde Pulte ignorieren sie anhand der Destination-UID.
- Die vier `DIR_IN`-Leitungen werden **zusammengefasst** auf GP21 geführt. Siehe Abschnitt 5 für die daraus folgenden RDM-Einschränkungen.
- `DIR_OUT` ist ebenfalls gemeinsam. Während eines RDM-Fensters sind daher **alle 16 Ausgänge** im Empfangsmodus und treiben kein DMX.

1.2 Anschluss der 74HC151

Beide Multiplexer teilen sich `A/B/C` (GP22/23/24). Kanal 0..7 wählt innerhalb einer Gruppe den Ausgang 1..8 bzw. 9..16:

```
OUT 1..8   -> 74HC151 #1 Eingänge D0..D7, Y1 -> GP26
OUT 9..16  -> 74HC151 #2 Eingänge D0..D7, Y2 -> GP27
A/B/C      -> GP22/GP23/GP24 (beide)
```

Die RDM-Rückleitungen der Ausgangs-Transceiver müssen auf die jeweiligen `D0..D7`-Eingänge gelegt werden. Die 74HC151-Ausgänge sind reine Logikpegel (3,3 V); die ADM2483-Ausgänge liefern bereits 3,3-V-kompatible Pegel, sofern die Logikseite mit 3,3 V versorgt wird.

1.3 ADM2483-Beschaltung

Die Transceiver-Beschaltung:

- **Eingangskarte:** 120 Ω fest zwischen A/B, keine Biasing-Widerstände, `RE`/`DE` zusammen auf `DIR_IN`, `TXD` auf GP20 (alle vier parallel).
- **Ausgangskarte:** keine feste Terminierung, 560 Ω Fail-Safe-Biasing, `RE`/`DE` zusammen auf `DIR_OUT`, `RXD` über den 74HC151 zurück.
- **Logikpegel:** Der ADM2483-Logikteil (`VDD1`) muss mit **3,3 V** betrieben werden, wenn `TxD`, `RxD` und `DIR` direkt an den RP2040 gehen. Bei `VDD1 = 5 V` liefert `RxD` 5-V-Pegel; der RP2040 ist nicht 5-V-tolerant. Die Busseite (`VDD2`) bleibt bei 5 V. Alternativ Pegelwandler vorsehen.
- SM712 an A/B, 10 Ω-Schutzwiderstände zu den XLR-Pins, isolierte B0505S-1W-Versorgung je Port.

Alle 16 Ausgangskarten teilen sich **einen** gemeinsamen `DIR_OUT`.

---

2. Architektur der Firmware

```
TX-Frame (513 Byte) ──> DmxRouter (HTP-Merge) ──> DmxTxGroup 0 (OUT 1..8)
                                                  └─> DmxTxGroup 1 (OUT 9..16)
```

```
GP16..19 ──> DmxInput 0..3 ──> DmxRouter
GP26/27 ──> DmxInput 4/5 (MUX) ──> RdmManager
```

2.1 Ausgänge: Bitplane statt 16 State Machines

Der RP2040 hat nur 8 PIO-State-Machines. Vier davon werden für die Eingänge benötigt, zwei für die RDM-Multiplexer. Für die 16 Ausgänge bleiben zwei SMs. Jede SM treibt acht **aufeinanderfolgende** GPIOs.

Ein DMA-Wort enthält vier „Bit-Slices“. Ein Slice ist ein Byte, in dem Bit *p* den Wert von Ausgang *p* zu einem Zeitpunkt enthält. Die PIO gibt pro Wort viermal `out pins, 8` aus (4 µs pro Slice). Damit werden acht DMX-Ports parallel erzeugt. Die Bit-Transponierung passiert in `DmxTxGroup::buildPlanes()`.

- Break: 48 Slices = **192 µs** (E1.11 ≥ 88 µs, E1.20 176..352 µs)
- MAB: 4 Slices = **16 µs**
- Nutzdaten: 513 Byte × 11 Bit (1 Start + 8 Daten + 2 Stop)
- Gesamtframe: ca. **22,8 ms** → maximal **43 Hz**
- Bei einem RDM-Request erhält nur der adressierte Port einen Break; die übrigen sieben Ports der Gruppe bleiben im Mark-Zustand und sehen keinen Phantom-Break.

Die konfigurierbare Refresh-Rate ist deshalb auf 5..43 Hz begrenzt. Standard ist 40 Hz.

2.2 Eingänge

Jeder Eingang hat eine eigene PIO-SM mit 8N2-UART-Empfänger. Der Empfänger liefert pro Zeichen neun Bits: Bit 8 ist das erste Stop-Bit. Ein Break wird als `0x000` erkannt und beendet den Frame.

- DMX-Frames mit weniger als 512 Slots werden beim nächsten Break abgeschlossen; fehlende Slots werden als 0 übernommen.
- Ein separater `_frame`-Puffer sorgt dafür, dass der Router nie einen halb empfangenen Frame weiterverarbeitet.
- Nur Startcode `0x00` wird in den HTP-Merge einbezogen. Andere Startcodes (z. B. Text/Test) werden ignoriert.
- RDM-Frames (`0xCC 0x01 ...`) landen in einer 2er-Warteschlange pro Eingang. Discovery-Antworten (18 Byte, ohne Längenbyte) werden beim folgenden Break abgeschlossen.

2.3 Routing und HTP

Jeder Ausgang hat eine 4-Bit-Maske:

Bit	Quelle
0	IN1
1	IN2
2	IN3
3	IN4

- Maske `0` = **Hold last look**: der Ausgang behält seinen letzten Frame.
- Sonst wird pro Slot das Maximum aller aktiven, maskierten Eingänge ausgegeben (HTP).

- Ein Eingang gilt nach 1000 ms ohne neuen Frame als inaktiv.
- Der Merge läuft bei jedem neuen Eingangsframe sowie alle 200 ms (Timeout-Auswertung, Konfigurationsänderungen).

Standardbelegung (4-fach-Splitter):

```
OUT 1..4   <- IN1
OUT 5..8   <- IN2
OUT 9..12  <- IN3
OUT 13..16 <- IN4
```

---

3. Bauen und Flashen

3.1 Standalone

```
pio run -d DMX-Router
pio run -d DMX-Router -t upload
pio device monitor -d DMX-Router
```

Der erste Flashvorgang benötigt BOOTSEL. Danach funktioniert der Upload über den nativen 1200-bps-USB-Touch (`upload_protocol = picotool`).

3.2 Aus dem Repository-Root

```
pio run -e dmxrouter
pio run -e dmxrouter -t upload
make build-dmxrouter
make upload-dmxrouter
make monitor-dmxrouter
```

Der Root-Build verwendet `scripts/redirect_env.py`, um die Quellen aus `DMX-Router/src` zu beziehen.

3.3 Build-Konfiguration

- Core: `earlephilhower` Arduino-Pico
- Board: `rpipico`
- Dateisystem: 64 KB LittleFS (`/router.cfg`)
- Keine externen Bibliotheken; SSD1306-Treiber und Font sind enthalten.

---

4. Bedienung

4.1 USB-Serial-Konsole (115200 Baud)

Nach dem Boot meldet sich die Firmware mit einer kurzen Zusammenfassung. `help` zeigt alle Befehle:

```
help                    this text
status                  counters, inputs, routing
route                   show routing table
route <out>             show one output (1..16)
route <out> <mask>      set one output
route all <mask>        set all outputs
refresh <hz>            refresh rate 5..43 Hz
rdm [status]            show RDM state
rdm on | off            enable/disable RDM relay
rdm test <out> <hex>    send a raw RDM wire frame
save                    write /router.cfg
defaults                restore defaults and save
reboot                  restart the RP2040
```

Masken:

- `0`, `hold`, `off` → Ausgang hält den letzten Frame
- `1..15` → Bitmaske (IN1 = Bit 0)
- `1+2`, `1,3`, `2+4` → anschauliche Eingangsliste

Beispiele:

```
route 5 1+2      # OUT5 = HTP(IN1, IN2)
route 1 1        # OUT1 = nur IN1
route all 15     # alle Eingänge HTP auf alle Ausgänge
route all 0      # alle Ausgänge halten
refresh 40
save
```

Änderungen über `route`, `refresh` und `rdm on|off` gelten sofort, werden aber erst mit `save` dauerhaft gespeichert. `defaults` setzt zurück und speichert direkt.

`rdm test` erwartet einen **kompletten RDM-Wire-Frame inklusive Prüfsumme** als Hex-String, z. B.:

```
rdm test 1 CC0100...
```

Der Frame wird zuerst mit `rdmValidate()` geprüft. Die Prüfsumme ist die **16-Bit-Summe aller Wire-Bytes einschließlich `0xCC`, ohne die beiden Prüfsummen-Bytes** (siehe Abschnitt 5).

4.2 OLED / Tasten (optional)

Verdrahtung:

- SSD1306 128×64 an I2C (0x3C, Fallback 0x3D)
- PCF8574 an I2C (0x20..0x27), Tasten P0..P3 gegen GND
- P0 = UP, P1 = DOWN, P2 = OK, P3 = BACK, active-low, 25 ms Entprellung

Bedienung:

Modus	UP/DOWN	OK	BACK
Status	Seite 1..8 / 9..16	Auswahl starten	RDM on/off
Auswahl	Ausgang wählen	Editieren	zurück
Edit	Maske erhöhen/verkleinern	speichern	abbrechen

Die Statusseite zeigt aktive Eingänge, RDM-Status, Refresh-Rate und die Routings der acht Ausgänge der aktuellen Seite. Die Anzeige wird alle 300 ms oder bei Änderungen neu gezeichnet.

Sind OLED oder PCF8574 nicht vorhanden, läuft der Router normal weiter; die Konfiguration erfolgt dann ausschließlich über die Serial-Konsole.

---

5. RDM: bewusste Einschränkungen

Die Hardware dieses Splitters kann **kein vollständiger RDM-Controller** sein. Die Firmware implementiert deshalb ein Store-and-Forward-Relay:

1. Auf einem Eingang kommt ein RDM-Request an (`0xCC 0x01 ...`).
2. Der Request wird an alle Ausgänge gesendet, deren Routing-Maske den Eingang enthält.
3. `DIR_OUT` wird auf Empfang geschaltet, der 74HC151 wählt den Ausgang, und der Rückkanal wird von einer PIO-UART gelesen.
4. Die erste gültige Antwort wird an den anfragenden Eingang weitergeleitet (UART1-TX mit 192-µs-Break).

Einschränkungen:

- **Ein gemeinsamer `DIR_OUT`:** Während des Antwortfensters sind alle 16 Ausgänge im Empfangsmodus. DMX wird in dieser Zeit nicht gesendet. Das ist bei einem RDM-Request auf einer DMX-Leitung normal, betrifft hier aber alle Ports gleichzeitig. Der Break wird nur auf dem adressierten Port ausgegeben; die übrigen sieben Ports der Gruppe bleiben während des Requests im Mark-Zustand.
- **Nur ein Transaktion zur Zeit:** Weitere Requests werden verworfen, solange eine Transaktion läuft. Pulte müssen ggf. retryen.
- **Nur der erste Responder:** Ein 74HC151 liest genau einen Kanal. Discovery-Kollisionen (mehrere Geräte antworten gleichzeitig) können nicht aufgelöst werden. Für Discovery ist das Relay daher nur eingeschränkt geeignet.
- **Antwortfenster 12 ms**, Abbruch nach 4 ms ohne Aktivität.
- **Gemeinsamer Eingangs-TX:** Eine Antwort geht an alle vier Pulte; fremde Pulte verwerfen sie über die Destination-UID.
- **Gemeinsamer Eingangs-DIR:** Während der Antwort sind alle vier Eingangs-Transceiver im Sendemodus. Pulte, die gleichzeitig DMX senden, können ihre eigene Leitung treiben; das ist bei RDM auf derselben Leitung ohnehin nicht zulässig.
- Kein vollständiges E1.20-Discovery-/Mute-/Un-Mute-Verfahren, keine Queue, keine priorisierten Antworten.

5.1 Prüfsumme

Die Firmware verwendet die **additive 16-Bit-Summe** über alle Wire-Bytes inklusive `0xCC`, aber **ohne** die beiden Checksum-Bytes:

```
checksum = (sum(wire[0 .. L-1]) & 0xFFFF)
```

Dabei ist `L = wire[2]` die Anzahl der Slots von `0xCC` bis zum letzten Parameterbyte. Die beiden Checksum-Bytes stehen bei `wire[L]` und `wire[L+1]`; die Gesamtlänge auf dem Draht ist `L + 2`. Minimale Länge: `L = 24`.

Die in manchen Projekten verwendete CRC-Implementierung (z. B. im `ArtNet-USB`-Projekt) ist für RDM **nicht** korrekt und wird hier nicht verwendet. Die additive Prüfsumme wurde mit echten DMXter-Paketen verifiziert.

Discovery-Antworten sind 18 Byte lang (`0xCC 0x01` + 12 Byte UID + 4 Byte Checksum) und haben kein Längenbyte. Sie werden strukturell akzeptiert; ihre Prüfsumme ist eine 32-Bit-Form und wird nicht nachgerechnet.

---

6. Konfigurationsdatei

LittleFS-Datei `/router.cfg`:

Feld	Typ	Bedeutung
`magic`	`uint32`	`0x52544D58` („RTMX“)
`version`	`uint16`	aktuell 1
`route[16]`	`uint8[16]`	4-Bit-Maske je Ausgang
`refreshHz`	`uint8`	5..43
`rdmEnabled`	`uint8`	0/1
`reserved[3]`	`uint8[3]`	0
`checksum`	`uint32`	FNV-1a über alle Felder davor

Bei ungültigem Magic, falscher Version, ungültiger Maske oder falscher Prüfsumme werden die Defaults geladen. Die Datei wird nur bei `save`, `defaults` oder einer UI-Speicherung geschrieben.

---

7. Dateien

Datei	Inhalt
`src/main.cpp`	Setup, Hauptloop, Frame-Scheduler, Serial-CLI
`src/board.h`	korrigierte Pin-Belegung und Konstanten
`src/config.*`	LittleFS-Konfiguration
`src/dmx_tx.pio`	PIO-Bitplane-Sender

Datei	Inhalt
`src/dmx_out.*`	DMA/Transponierung, Break/MAB, Busy-Verwaltung
`src/dmx_rx.pio`	PIO-8N2-UART-Empfänger
`src/dmx_in.*`	DMX/RDM-Parser, RDM-Queue, Doppelpuffer
`src/router.*`	HTP-Merge, Hold-last, Timeouts
`src/rdm.*`	RDM-Prüfsumme, Relay, Multiplexer-Transaktion
`src/ssd1306.*`	eigener SSD1306-Treiber
`src/font5x7.h`	5×7-Font
`src/ui.*`	Status-/Auswahl-/Edit-Seiten, PCF8574-Tasten

---

8. Test-Checkliste für den Aufbau

1. **Spannungsversorgung:** 3,3 V Logik, 5 V isolierte Ports prüfen, keine Kurzschlüsse zwischen Logik- und Isolationsmasse.
2. **Ausgänge:** Oszilloskop an OUT1; Break ≈ 192 µs, MAB ≈ 16 µs, danach 513 Byte mit 44 µs/Bit. Bei 40 Hz erscheint alle 25 ms ein Frame.
3. **Eingänge:** Pult an IN1; `status` muss `frames` hochzählen und `active yes` zeigen.
4. **Routing:** `route 5 1`, `save`; OUT5 muss die Daten von IN1 ausgeben.
5. **HTP:** Zwei Pulte auf IN1/IN2, `route 1 1+2`; pro Kanal muss der größere Wert erscheinen.
6. **Hold:** `route 1 0`; OUT1 behält den letzten Pegel.
7. **RDM:** `rdm status`, dann `rdm test 1 <gültiger Frame>`; Antwort erscheint auf der Konsole. Ohne Responder: `no response` nach ca. 12 ms.
8. **UI:** OLED zeigt Status; Tasten ändern Routings und speichern.
9. **Reboot:** Nach `save` und Neustart müssen Routings und Refresh-Rate erhalten bleiben.

Bekannte offene Punkte:

- Hardware-Test steht noch aus (siehe Status oben).
- Während der RDM-Antwortweiterleitung (maximal ca. 12 ms bei 260 Byte) sind die vier Eingangs-Parser stummgeschaltet und werden nur noch geleert. Ein DMX-Frame, der genau in dieses Fenster fällt, geht verloren; der Parser resynchronisiert beim nächsten Break. Das ist nötig, weil die Halbduplex-Transceiver die eigene Antwort auf allen Eingängen mithören und sonst als neuen Request interpretieren würden.
- Die Onboard-LED an GP25 wird als `DIR_OUT` mitbenutzt.